

LabReSiD25 - Hands-On 2

Davide Ferrara - 518629

Traccia

Usando i Linux Socket scrivere un server bloccante (`server_tcp_block.c`), un server non bloccante (`server_tcp_nonblock.c`) ed un client (`client_tcp.c`) che realizzino una comunicazione basata su TCP. Il client deve inviare la successione dei numeri di Fibonacci, mentre il server deve effettuare un controllo sulla successione ed inviare una stringa di acknowledgment (attenzione, diverso da ACK!).

1 Client TCP

1.1 Obiettivo

Il client genera la sequenza di Fibonacci fino a $F(n)$, la converte in stringa (es. "0 1 1 2 3 5\n") e la invia al server via TCP. Attende la risposta (ACK o NACK) e ripete per `reps` volte sulla stessa connessione prima di chiudere. Accetta due argomenti da riga di comando: `n` (indice di Fibonacci) e `reps` (numero di invii, default 1).

1.2 client.c

1.2.1 Include e costanti

`SERVER_ADDR` è impostato a 0.0.0.0 per connettersi al localhost. `int_to_char()` converte un intero in carattere ASCII sommando '0', operazione inversa rispetto al parsing del server.

```
1 #include "fibonacci.h"
2 #include <arpa/inet.h>
3 #include <netinet/in.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <string.h>
```

```

7 #include <sys/socket.h>
8 #include <unistd.h>
9
10 #define SERVER_ADDR "0.0.0.0"
11 #define PORT 8080
12
13 char int_to_char(int n) { return '0' + n; }

```

Listing 1: client.c - Include e costanti

1.2.2 Costruzione del messaggio

Genera la sequenza con `fibo_sequence()` e la converte in stringa cifra per cifra usando `int_to_char()`. I numeri sono separati da spazi e la stringa termina con `\n`.

```

1 int n = atoi(argv[1]);
2 int reps = (argc == 3) ? atoi(argv[2]) : 1;
3
4 // Genera la sequenza di Fibonacci fino a F(n) e la
5 // converte in stringa
6 // Es. n=5 -> "0 1 1 2 3 5\n"
6 fibo_sequence_t fs;
7 fibo_init(&fs);
8 fibo_sequence(&fs, n);
9 char msg[1024];
10 int cur = 0;
11 for (int i = 0; i < fs.idx; i++) {
12     if (i > 0) msg[cur++] = ' ';
13     msg[cur++] = int_to_char(fs.array[i]);
14 }
15 msg[cur++] = '\n';
16 msg[cur] = '\0';

```

Listing 2: client.c - costruzione messaggio

1.2.3 Connessione al server

`socket()` crea il socket TCP. `connect()` stabilisce la connessione con il server eseguendo il three-way handshake. La connessione viene aperta una sola volta prima del loop.

```

1 // SOCK_STREAM crea un socket TCP orientato alla
   // connessione

```

```

2 int client_fd = socket(AF_INET, SOCK_STREAM, 0);
3 if (client_fd == -1) {
4     perror("Errore nella creazione del socket");
5     exit(1);
6 }
7
8 // connect() stabilisce la connessione TCP con il server
   (three-way handshake)
9 if (connect(client_fd, (struct sockaddr *)&server_addr,
10             sizeof(server_addr)) == -1) {
11     perror("Errore nella connessione");
12     exit(1);
13 }

```

Listing 3: client.c - socket() e connect()

1.2.4 Loop di invio e ricezione

Il loop invia il messaggio `reps` volte sulla stessa connessione e attende la risposta del server ad ogni iterazione. `close()` chiude la connessione solo dopo l'ultimo invio.

```

1 // Invia il messaggio reps volte sulla stessa
   connessione e attende ACK/NACK
2 for (int i = 0; i < reps; i++) {
3     if (write(client_fd, msg, strlen(msg)) == -1) {
4         perror("Errore nel socket");
5         exit(1);
6     }
7
8     char buf[1024];
9     int r = read(client_fd, buf, sizeof(buf));
10    buf[r] = '\0';
11    printf("[CLIENT.c] %s:%d: %s", SERVER_ADDR, PORT, buf)
        ;
12
13    sleep(1);
14 }
15
16 // Chiude la connessione dopo aver inviato tutti i
   messaggi
17 close(client_fd);

```

Listing 4: client.c - loop write/read

1.3 Output di esempio

```
[CLIENT.c] Sequenza di fibonacci: 0 1 1 2 3 5
[CLIENT.c] 0.0.0.0:8080: ACK
[CLIENT.c] 0.0.0.0:8080: ACK
[CLIENT.c] 0.0.0.0:8080: ACK
```

Listing 5: Output client - ./client 5 3

2 Server TCP Bloccante

2.1 Obiettivo

Il server TCP bloccante riceve dal client una sequenza di Fibonacci sotto forma di stringa (es. "0 1 1 2 3"), legge il messaggio dalla socket, effettua il parsing convertendo ogni carattere nel corrispondente intero e ricostruisce la sequenza. Se la sequenza è corretta risponde con la stringa **ACK**, altrimenti con **NACK**.

Essendo bloccante, il server non può gestire più di un client alla volta: le chiamate `accept()` e `read()` bloccano il processo finché non arriva una connessione o dei dati.

2.2 server_tcp_block.c

2.2.1 Include e costanti

Includo le librerie necessarie. Rispetto a UDP aggiungo `netinet/in.h` per le strutture TCP.

```
1 #include "fibo.h"
2 #include <arpa/inet.h>
3 #include <netinet/in.h>
4 #include <stdio.h>
5 #include <stdlib.h>
6 #include <string.h>
7 #include <sys/socket.h>
8 #include <unistd.h>
9
10 #define PORT 8080
```

Listing 6: server_tcp_block.c - Include

2.2.2 parse_msg

Parsa il messaggio ricevuto dal client e popola la struttura `fibonacci_sequence_t`. I numeri sono separati da spazi. Caratteri `\n` e `\r` vengono ignorati. La conversione da carattere a intero sfrutta la tabella ASCII: le cifre `'0' ... '9'` sono consecutive (48...57), quindi `'7' - '0' = 55 - 48 = 7`.

```
1 void parse_msg(char *msg, fibonacci_sequence_t *fs) {
2     fibo_init(fs);
3
4     while (*msg != '\0') {
5         if (*msg == ' ' || *msg == '\n' || *msg == '\r') {
6             msg++;
7             continue;
8         }
9
10        int n = *msg - '0';
11        fibo_push(fs, n);
12
13        msg++;
14    }
15 }
```

Listing 7: server_tcp_block.c - parse_msg()

2.2.3 Creazione del socket

Creo un socket di tipo `SOCK_STREAM` per TCP. La differenza con UDP (`SOCK_DGRAM`) è fondamentale poichè `SOCK_STREAM` garantisce consegna ordinata e affidabile dei dati tramite connessione.

`SO_REUSEADDR` permette di riutilizzare la porta subito dopo la chiusura del server, evitando l'errore `Address already in use` causato dallo stato `TIME_WAIT` di TCP.

```
1 struct sockaddr_in addr;
2 addr.sin_family = AF_INET;
3 addr.sin_port = htons(PORT);
4 addr.sin_addr.s_addr = INADDR_ANY;
5
6 // SOCK_STREAM crea un socket TCP orientato alla
7 // connessione
8 int server_fd = socket(AF_INET, SOCK_STREAM, 0);
9 if (server_fd == -1) {
10     perror("Errore nella creazione del socket");
11 }
```

```

10  exit(1);
11  }
12
13  // Evita "Address already in use" al riavvio: TCP tiene
    il socket
14  // in TIME_WAIT per ~2 minuti dopo la chiusura
15  int opt = 1;
16  setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt,
    sizeof(opt));

```

Listing 8: server_tcp_block.c - socket() e SO_REUSEADDR

2.2.4 bind() e listen()

bind() associa il socket all'indirizzo e porta configurati. listen() mette il socket in modalità ascolto con un backlog di 5: il kernel può tenere in coda fino a 5 connessioni in attesa di essere accettate. **Nota:** listen() non è bloccante, ritorna subito.

```

1  if (bind(server_fd, (struct sockaddr *)&addr, sizeof(
    addr)) == -1) {
2      perror("Errore nel bind del socket");
3      exit(1);
4  }
5
6  // Il backlog di 5 indica quante connessioni il kernel
    puo' accodare
7  // mentre il server e' occupato a servire un client
8  if (listen(server_fd, 5) == -1) {
9      perror("Error during listening");
10 }

```

Listing 9: server_tcp_block.c - bind() e listen()

2.2.5 Loop di accettazione: accept()

Il loop esterno chiama accept(), che **blocca** il processo finché un client non si connette. Quando arriva una connessione, accept() ritorna un nuovo file descriptor (client_fd) dedicato esclusivamente a quella connessione. Il server_fd originale rimane aperto per accettare altri client. accept() popola anche la struttura client_addr con IP e porta del client.

```

1  struct sockaddr_in client_addr;
2  socklen_t addrlen = sizeof(client_addr);

```

```

3 char buf[1024];
4
5 // Loop esterno: il server non termina mai, accetta un
  client alla volta
6 while (1) {
7     // accept() blocca finche' un client non si connette.
8     // Ritorna un nuovo fd dedicato a quella connessione.
9     // Finche' siamo qui dentro, nessun altro client viene
    accettato.
10    int client_fd =
11        accept(server_fd, (struct sockaddr *)&client_addr,
            &addrlen);
12    if (client_fd == -1) {
13        perror("Could not accept connection");
14        continue;
15    }
16    // loop interno ...
17 }

```

Listing 10: server_tcp_block.c - accept()

2.2.6 Loop di lettura: read()

Il loop interno chiama `read()`, che **blocca** finché non arrivano dati. Il valore di ritorno indica:

- `n > 0`: bytes letti correttamente
- `n == 0`: il client ha chiuso la connessione
- `n == -1`: errore

```

1 // Loop interno: legge piu' messaggi dalla stessa
  connessione
2 // finche' il client non si disconnette
3 while (1) {
4     // read() blocca finche' non arrivano dati.
5     // Ritorna 0 quando il client chiude la connessione (
    EOF).
6     ssize_t n = read(client_fd, buf, sizeof(buf));
7     if (n == 0) {
8         printf("Il Client si e' disconnesso!\n");
9         close(client_fd);
10        break;

```

```

11 }
12 if (n == -1) {
13     printf("Errore del client!\n");
14     break;
15 }
16
17 char ip_str[INET_ADDRSTRLEN];
18 inet_ntop(AF_INET, &client_addr.sin_addr, ip_str,
19           sizeof(ip_str));
20 uint16_t port = ntohs(client_addr.sin_port);
21
22 buf[n] = '\0'; // aggiunge terminatore per usare buf
23               // come stringa C
24 printf("[SERVER_TPC_BLOCK.c] %s:%d: %s", ip_str, port,
25       buf);
26 // verifica fibonacci ...
27 }

```

Listing 11: server_tcp_block.c - read()

2.2.7 Verifica Fibonacci e risposta

Dopo aver ricevuto il messaggio, il server:

1. Parsa la sequenza ricevuta dal client con `parse_msg()`
2. Genera la propria sequenza con `fibo_sequence()` della stessa lunghezza
3. Confronta le due sequenze con `fibo_compare()`
4. Invia ACK se corretta, NACK se sbagliata

```

1 fibo_sequence_t fs, client_fs;
2 memset(&fs, 0, sizeof(fibo_sequence_t));
3 fibo_init(&fs);
4
5 // Parsa la stringa ricevuta e genera la sequenza attesa
6 parse_msg(buf, &client_fs);
7 fibo_sequence(&fs, client_fs.idx - 1);
8
9 if (fibo_compare(&fs, &client_fs) == 0) {
10     char *resp = "ACK\n";
11     write(client_fd, resp, strlen(resp));

```



```

12 } else {
13     char *resp = "NACK\n";
14     write(client_fd, resp, strlen(resp));
15 }
16 // non chiudiamo: il client puo' inviare altri messaggi

```

Listing 12: server_tcp_block.c - verifica e risposta

2.3 Limitazione: un client alla volta

Essendo bloccante, il server non può servire più client contemporaneamente. Mentre il loop interno legge e risponde ai messaggi del client A (porta 57804), la chiamata `accept()` non viene mai rieseguita: il client B (porta 57820) rimane in attesa nel backlog del kernel. Solo dopo che A si disconnette (`read()` ritorna 0), il server chiude `client_fd`, esce dal loop interno e torna ad `accept()`, servendo B.

2.4 Output di esempio

```

[TPC SERVER BLOCKING] Server is listening...
[SERVER_TPC_BLOCK.c] 127.0.0.1:57804: 0 1 1 2 3 5
[SERVER_TPC_BLOCK.c] Sent response to 127.0.0.1:57804:
    ACK
[SERVER_TPC_BLOCK.c] 127.0.0.1:57804: 0 1 1 2 3 5
[SERVER_TPC_BLOCK.c] Sent response to 127.0.0.1:57804:
    ACK
[SERVER_TPC_BLOCK.c] 127.0.0.1:57804: 0 1 1 2 3 5
[SERVER_TPC_BLOCK.c] Sent response to 127.0.0.1:57804:
    ACK
Il Client si e' disconnesso!
[SERVER_TPC_BLOCK.c] 127.0.0.1:57820: 0 1 1 2 3 5
[SERVER_TPC_BLOCK.c] Sent response to 127.0.0.1:57820:
    ACK
[SERVER_TPC_BLOCK.c] 127.0.0.1:57820: 0 1 1 2 3 5
[SERVER_TPC_BLOCK.c] Sent response to 127.0.0.1:57820:
    ACK
Il Client si e' disconnesso!

```

Listing 13: Output server - due client sequenziali

```

[CLIENT.c] Sequenza di fibonacci: 0 1 1 2 3 5
[CLIENT.c] 0.0.0.0:8080: ACK

```

```
[CLIENT.c] 0.0.0.0:8080: ACK
[CLIENT.c] 0.0.0.0:8080: ACK
```

Listing 14: Output client - ./client 5 3

3 Server TCP Non Bloccante

3.1 Obbiettivo

Il server TCP non bloccante riceve dal client una sequenza di Fibonacci sotto forma di stringa (es. "0 1 1 2 3"), legge il messaggio dalla socket, effettua il parsing convertendo ogni carattere nel corrispondente intero e ricostruisce la sequenza. Se la sequenza è corretta risponde con la stringa **ACK**, altrimenti con **NACK**.

A differenza del server bloccante, le chiamate `accept()` e `read()` non bloccano il processo: ritornano immediatamente con **EAGAIN** se non ci sono dati disponibili. Il processo continua a girare nel loop invece di dormire, realizzando una **attesa attiva** (busy-waiting).

3.2 server_tcp_nonblock.c

3.2.1 Include e costanti

Rispetto al server bloccante aggiungo `errno.h` per leggere il codice di errore dopo le chiamate non bloccanti, e `fcntl.h` per la funzione `fcntl()` che imposta il flag `O_NONBLOCK`.

```
1 #include "fibonacci.h"
2 #include <arpa/inet.h>
3 #include <errno.h>
4 #include <fcntl.h>
5 #include <netinet/in.h>
6 #include <stdio.h>
7 #include <stdlib.h>
8 #include <string.h>
9 #include <sys/socket.h>
10 #include <unistd.h>
11
12 #define PORT 8080
```

Listing 15: server_tcp_nonblock.c - Include

3.2.2 set_nonblocking()

`fcntl()` legge prima i flag attuali del file descriptor con `F_GETFL`, poi li riscrive aggiungendo `O_NONBLOCK` con `F_SETFL`. Da quel momento, qualsiasi chiamata bloccante su quel fd ritorna immediatamente con `errno = EAGAIN` se non ci sono dati.

```
1 /*
2  * Imposta il flag O_NONBLOCK sul file descriptor fd
3  * tramite fcntl().
4  * Dopo questa chiamata, accept() e read() su fd
5  * ritornano subito con
6  * errno = EAGAIN invece di bloccare il processo in
7  * attesa di dati.
8  */
9 static void set_nonblocking(int fd) {
10     int flags = fcntl(fd, F_GETFL, 0);
11     fcntl(fd, F_SETFL, flags | O_NONBLOCK);
12 }
```

Listing 16: server_tcp_nonblock.c - set_nonblocking()

3.2.3 parse_msg

Identica al server bloccante: parse la stringa ricevuta e popola la struttura `fibonacci_sequence_t`. I numeri sono separati da spazi e la conversione sfrutta la tabella ASCII ('7' - '0' = 7).

```
1 void parse_msg(char *msg, fibonacci_sequence_t *fs) {
2     fibo_init(fs);
3
4     while (*msg != '\0') {
5         if (*msg == ' ' || *msg == '\n' || *msg == '\r') {
6             msg++;
7             continue;
8         }
9
10        int n = *msg - '0';
11        fibo_push(fs, n);
12
13        msg++;
14    }
15 }
```

Listing 17: server_tcp_nonblock.c - parse_msg()

3.2.4 Creazione del socket e O_NONBLOCK

La creazione del socket è identica al server bloccante. La differenza è la chiamata a `set_nonblocking(server_fd)` prima del `bind()`: da quel momento `accept()` non dormirà mai, ma ritornerà immediatamente.

```
1 // SOCK_STREAM crea un socket TCP orientato alla
   connessione
2 int server_fd = socket(AF_INET, SOCK_STREAM, 0);
3 if (server_fd == -1) {
4     perror("Errore nella creazione del socket");
5     exit(1);
6 }
7
8 // Evita "Address already in use" al riavvio: TCP tiene
   il socket
9 // in TIME_WAIT per ~2 minuti dopo la chiusura
10 int opt = 1;
11 setsockopt(server_fd, SOL_SOCKET, SO_REUSEADDR, &opt,
   sizeof(opt));
12
13 // Rende il server fd non bloccante: accept() ritorna
   EAGAIN invece di
14 // bloccare il processo quando non ci sono connessioni
   in arrivo
15 set_nonblocking(server_fd);
```

Listing 18: server_tcp_nonblock.c - `socket()` e `O_NONBLOCK`

3.2.5 Loop esterno: `accept()` con attesa attiva

Il loop chiama `accept()` che ritorna subito. Se `client_fd < 0` e `errno == EAGAIN` significa che non c'è nessun client in coda: il loop continua immediatamente senza dormire. Appena arriva una connessione, anche il `client_fd` viene reso non bloccante con `set_nonblocking()`.

```
1 // Loop esterno con attesa attiva: il processo non si
   blocca mai,
2 // gira continuamente controllando se arriva una
   connessione
3 while (1) {
4     int client_fd =
5         accept(server_fd, (struct sockaddr *)&client_addr,
           &addrlen);
```

```

6
7  if (client_fd < 0) {
8      if (errno == EWOULDBLOCK || errno == EAGAIN) {
9          // Nessuna connessione in arrivo: continua il loop
           senza bloccarsi
10         continue;
11     } else {
12         perror("Errore accept");
13         exit(1);
14     }
15 }
16
17 printf("[SERVER_TCP_NONBLOCK.c] Nuovo client connesso\
18         n");
19
20 // Rende anche il client fd non bloccante: read()
   ritornera' EAGAIN
21 // invece di bloccare il processo quando non ci sono
   dati
22 set_nonblocking(client_fd);
23 // loop interno ...
24 }

```

Listing 19: server_tcp_nonblock.c - accept() con attesa attiva

3.2.6 Loop interno: read() con attesa attiva

read() ritorna immediatamente. Il valore di ritorno indica:

- $n > 0$: bytes letti correttamente
- $n == 0$: il client ha chiuso la connessione (EOF)
- $n < 0$ con `errno == EAGAIN`: nessun dato ora, riprova
- $n < 0$ altro errore: errore reale, chiudi la connessione

```

1 // Loop interno con attesa attiva: legge piu' messaggi
   dalla stessa
2 // connessione finche' il client non si disconnette
3 while (1) {
4     // read() ritorna subito: EAGAIN se non ci sono dati,
5     // 0 se il client ha chiuso la connessione (EOF)
6     ssize_t n = read(client_fd, buf, sizeof(buf) - 1);

```

```

7
8  if (n < 0) {
9      if (errno == EWOULDBLOCK || errno == EAGAIN) {
10         // Nessun dato disponibile ora: continua il loop
           senza bloccarsi
11         continue;
12     }
13     perror("Errore read");
14     break;
15 }
16
17 if (n == 0) {
18     printf("Il Client si e' disconnesso!\n");
19     close(client_fd);
20     break;
21 }
22
23 buf[n] = '\0'; // aggiunge terminatore per usare buf
           come stringa C
24 printf("[SERVER_TCP_NONBLOCK.c] %s:%d: %s", ip_str,
           port, buf);
25 // verifica fibonacci ...
26 }

```

Listing 20: server_tcp_nonblock.c - read() con attesa attiva

3.2.7 Verifica Fibonacci e risposta

Identica al server bloccante: parse la sequenza ricevuta, genera quella attesa e confronta. Invia ACK se corretta, NACK altrimenti.

```

1  fibo_sequence_t fs, client_fs;
2  memset(&fs, 0, sizeof(fibo_sequence_t));
3  fibo_init(&fs);
4
5  // Parsa la stringa ricevuta e genera la sequenza attesa
6  parse_msg(buf, &client_fs);
7  fibo_sequence(&fs, client_fs.idx - 1);
8
9  if (fibo_compare(&fs, &client_fs) == 0) {
10     char *resp = "ACK\n";
11     write(client_fd, resp, strlen(resp));
12     printf("[SERVER_TCP_NONBLOCK.c] Sent response to %s:%d
           : %s", ip_str,

```

```

13         port, resp);
14 } else {
15     char *resp = "NACK\n";
16     write(client_fd, resp, strlen(resp));
17     printf("[SERVER_TCP_NONBLOCK.c] Sent response to %s:%d
18           : %s", ip_str,
19               port, resp);
20 }
21 // non chiudiamo: il client puo' inviare altri messaggi

```

Listing 21: server_tcp_nonblock.c - verifica e risposta

3.3 Limitazione: ancora un client alla volta

Nonostante le chiamate siano non bloccanti, il comportamento verso i client è identico al server bloccante: il loop interno gira su `read()` finché il client non si disconnette, e solo allora il loop esterno torna ad `accept()`. Il client B deve aspettare che il client A si disconnetta.

La differenza reale è che il processo non **dorme** mai: spende il 100% della CPU girando nel loop invece di andare in sleep come farebbe il server bloccante. Questa tecnica, chiamata **attesa attiva** (busy-waiting), è più reattiva ma inefficiente in termini di consumo CPU.

È possibile verificarlo con `top` mentre entrambi i server sono in esecuzione senza client connessi:

```

top -p $(pgrep -f server_tcp_block),$(pgrep -f
server_tcp_nonblock)

```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM
216495	dff	20	0	2556	1460	1364	R	92.2	0.0
		0:37.02		server_tcp_nonb					
217176	dff	20	0	2556	1460	1364	S	0.0	0.0
		0:00.00		server_tcp_bloc					

Listing 22: top -p \$(pgrep -f server_tcp_nonblock) - confronto CPU

Il server non bloccante è nello stato **R** (Running) al 92% di CPU, il server bloccante è nello stato **S** (Sleeping) allo 0%: il kernel lo ha sospeso in attesa di una connessione.

3.4 Output di esempio

```

[SERVER_TCP_NONBLOCK.c] Server is listening...
[SERVER_TCP_NONBLOCK.c] Nuovo client connesso
[SERVER_TCP_NONBLOCK.c] 127.0.0.1:40360: 0 1 1 2 3 5
[SERVER_TCP_NONBLOCK.c] Sent response to
    127.0.0.1:40360: ACK
[SERVER_TCP_NONBLOCK.c] 127.0.0.1:40360: 0 1 1 2 3 5
[SERVER_TCP_NONBLOCK.c] Sent response to
    127.0.0.1:40360: ACK
[SERVER_TCP_NONBLOCK.c] 127.0.0.1:40360: 0 1 1 2 3 5
[SERVER_TCP_NONBLOCK.c] Sent response to
    127.0.0.1:40360: ACK
Il Client si e' disconnesso!
[SERVER_TCP_NONBLOCK.c] Nuovo client connesso
[SERVER_TCP_NONBLOCK.c] 127.0.0.1:40372: 0 1 1 2 3 5
[SERVER_TCP_NONBLOCK.c] Sent response to
    127.0.0.1:40372: ACK
Il Client si e' disconnesso!

```

Listing 23: Output server - due client sequenziali

```

[CLIENT.c] Sequenza di fibonacci: 0 1 1 2 3 5
[CLIENT.c] 0.0.0.0:8080: ACK
[CLIENT.c] 0.0.0.0:8080: ACK
[CLIENT.c] 0.0.0.0:8080: ACK

```

Listing 24: Output client A - ./client 5 3

```

[CLIENT.c] Sequenza di fibonacci: 0 1 1 2 3 5
[CLIENT.c] 0.0.0.0:8080: ACK

```

Listing 25: Output client B - ./client 5 1